NLP

11-06-26

Issues with BOW approach for text classification:

- word frequency is not being used
- exclusive/unique words are not given more importance
- order of the words is not captured

eg. vocab: [serve, good, food, bad, not, park]

1. food is not bad and service is good.

↓

[food, not, bad, serve, good] → [1, 1, 1, 1, 1, 0] ✓

2. food is not good and service is bad

↓

[food, not, good, serve, bad] → [1, 1, 1, 1, 1, 0] ✓

- Out of vocabulary word/tokens are not represented at all in vector form.

(this also causes zero probability issue in Naive Bayes model)

First two issues are solved using TFIdf approach

TF - Term frequency

- gives more weight to words having more frequency within same document

IDF - Inverse Document Frequency

- gives more importance to exclusive words

Order of words is ignored in TFIDF too!

There are some other issues with frequency and vocabulary based methods

1. The size of vector representing the document depends on vocab size
   - if vocab size is very large, vector size will also be very large
   - most of the values in vector will be zero

eg: vocab size: 20,000

food is nice → [food, nice] → [0, 0, 0, 0 ... 1 .. 1, 0 0]

19998 → 0
1 → 2

such big and sparse vectors as input are not good for the model training

solution of this → encoding of words

In vocab, assign a unique token id to every token

vocab: {'a': 1, 'ant': 2, 'age': 3, ... 'india': 10425, ... 'zebra': 20000}

vocab: dictionary - pair {token: token-id}

text: I live in India
vector: [9001, 12845, 9050, 10425]

with encoding of tokens, vector size now depends on length of text and not on vocabulary size.



Text classification using ANN:

eg. Text	sentiment
This car is very safe	+ve
interiors are really great	+ve
Too good!	+ve
boot space is small, not ideal for family of 4	-ve
sunroof is small	-ve
⋮	⋮

1000 reviews

↓ vocabulary

{'a': 1, 'am': 2, 'about': 3 . . . . 'you': 5600}

This car is very safe

↓ tokenization

[101, 105, 1000, 1020, 4222] token ids

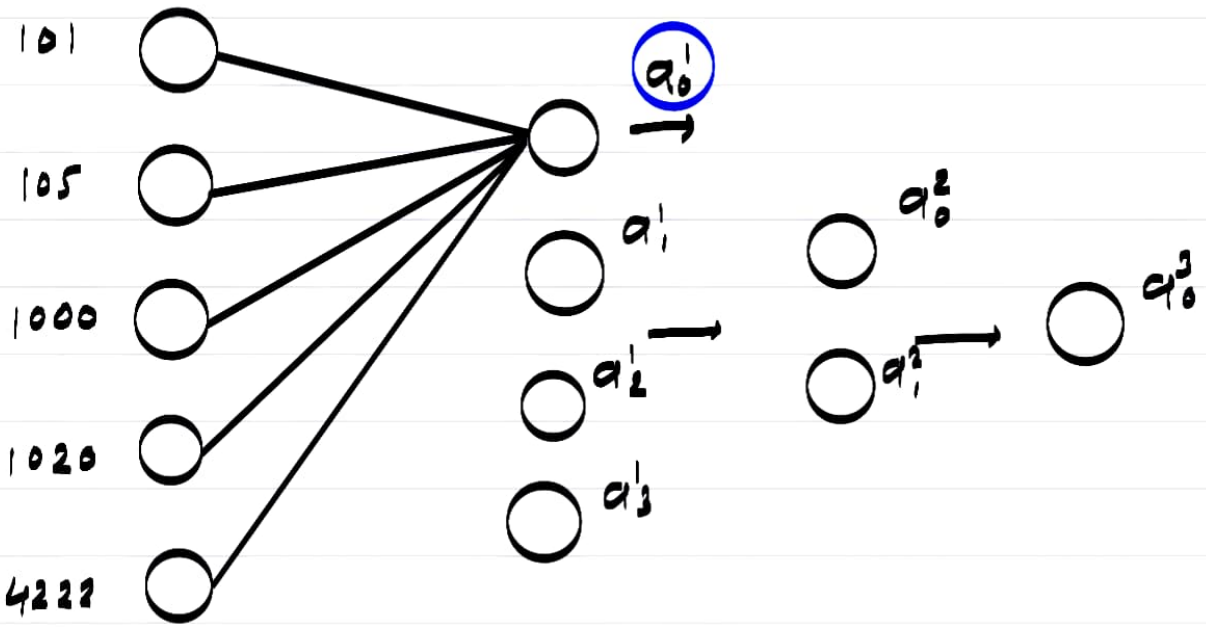
↓

ANN → +ve/-ve

This car is very safe

↓ tokenization

[101, 105, 1000, 1020, 4222] token ids



in layer 1

$$z_0^1 = w_0^1 a^0 = w_0^1 x$$

$$z_0^1 = w_{00}^1 (101) + w_{01}^1 (105) + w_{02}^1 (1000) + w_{03}^1 (1020) + w_{04}^1 (4222)$$

$$a_0^1 = f_1(z_0^1)$$

$$z_1^1 = w_1^1 a^0 = w_1^1 x$$

$$z_1^1 = w_{10}^1 (101) + w_{11}^1 (105) + w_{12}^1 (1000) + w_{13}^1 (1020) + w_{14}^1 (4222)$$

$$a_1^1 = f_1(z_1^1)$$

⋮

This ANN model can only take 5 tokens as input as there are only 5 nodes in input layer

What happens if no. of tokens is < 5 or > 5?

1. car is awsome

↓ tokenization

[545, 946, 105] + padding

↓

[545, 946, 105, 0, 0]

2. seat leather is high quality. There are 4 airbags.

↓ tokenization

[841, 246, 10, 107, 1047, 911, 234, 67, 122] 9 token

↓ truncation

[841, 246, 10, 107, 1047]

Padding: Append 0s at the end to match designed sequence length

Truncation: Truncate/remove token ids beyond designed sequence length

In real world designed sequence lengths are chosen like - 512, 1024, 2048, 784

↓

BERT transformer sequence length

- ANN processes all tokens of sequence at once and the input tokens are fully connected to hidden layers

- Due to such processing, ANN can't capture complete order and context of tokens

solution : A new ANN architecture specialised for sequence inputs called RNN

Recurrent Neural Network

- RNN is specialised ANN for sequential inputs

- in RNN tokens are processed in same order as they appear in text hence model becomes aware of the order of words

- RNN is also able to learn dependencies among the words

- RNN works with variable sequence lengths hence padding or truncation are not required.